

CIELAB to RGB Conversion Guide

A technical reference for transforming colors from the CIELAB color space to standard sRGB.

To convert colors from **CIELAB (L*a*b*)** to **RGB**, you must use a two-step mathematical process because there is no direct formula between them. First, convert the CIELAB values to the intermediate **CIE XYZ color space**, and then transform those XYZ values into the target **RGB color space** (usually standard sRGB). This guide assumes a standard **D65 illuminant** reference white point.

Step 1: Convert CIELAB to CIE XYZ

Given the input values L* (Lightness, 0 to 100), a* (green to red, approx. -128 to 127), and b* (blue to yellow, approx. -128 to 127):

1. Calculate intermediate variables (f_x, f_y, f_z):

$$f_y = (L^* + 16) / 116$$

$$f_x = (a^* / 500) + f_y$$

$$f_z = f_y - (b^* / 200)$$

2. Apply the inverse CIE transformation function:

For each f in $\{f_x, f_y, f_z\}$, calculate the corresponding value x, y , or z :

$$\text{If } f > \delta: f^3$$

$$\text{Otherwise: } 3\delta^2 \times (f - 4/29)$$

(Where $\delta = 6/29$, meaning $\delta^2 \approx 0.0428$ and $3\delta^2 \approx 0.1284$)

3. Scale by the Reference White Point (X_n, Y_n, Z_n):

Using the standard D65 white point normalized to $Y=1$ ($X_n = 0.950489$, $Y_n = 1.000000$, $Z_n = 1.088840$):

$$X = x \times X_n$$

$$Y = y \times Y_n$$

$$Z = z \times Z_n$$

Step 2: Convert CIE XYZ to sRGB

With your X, Y, and Z values, you can now transform them into standard RGB channels.

1. Apply the Transformation Matrix:

Multiply the XYZ vector by the standard D65 sRGB transformation matrix:

$$R_{\text{linear}} = 3.2406 \times X - 1.5372 \times Y - 0.4986 \times Z$$

$$G_{\text{linear}} = -0.9689 \times X + 1.8758 \times Y + 0.0415 \times Z$$

$$B_{\text{linear}} = 0.0557 \times X - 2.0402 \times Y + 1.0570 \times Z$$

2. Apply Gamma Correction (Compandation):

To convert linear RGB (C_{linear}) to sRGB (C_{srgb}), apply the following to R, G, and B:

$$\text{If } C_{\text{linear}} \leq 0.0031308 : C_{\text{srgb}} = 12.92 \times C_{\text{linear}}$$

$$\text{Otherwise : } C_{\text{srgb}} = 1.055 \times (C_{\text{linear}})^{(1/2.4)} - 0.055$$

3. Scale and Clamp to 8-bit Integer (0-255):

$$R = \max(0, \min(255, \text{round}(R_{\text{srgb}} \times 255)))$$

$$G = \max(0, \min(255, \text{round}(G_{\text{srgb}} \times 255)))$$

$$B = \max(0, \min(255, \text{round}(B_{\text{srgb}} \times 255)))$$

Implementation Examples

Instead of manual calculations, code implementations or libraries can handle this conversion automatically.

Python Example (scikit-image):

```
import skimage.color as color
# Input LAB values: L=53, a=80, b=67
rgb = color.lab2rgb([[[53, 80, 67]]]) * 255
```

MATLAB Example:

```
rgb = lab2rgb(lab);
```